

Intermediate Report — Robot-Ready DeiT-Tiny (DeiT, 2012.12877)

Paper: Data-Efficient Image Transformers (DeiT)

Local PDF (for reference): /mnt/data/2012.12877v2.pdf

Repository: <https://github.com/Grandlew/MLDMproject>

Objective

Re-implement a small DeiT/ViT model and validate it on a lightweight segmentation proxy for robot traversability (toy sample now; Freiburg Forest slice next). The focus is on understanding, clean re-implementation, and reproducible training/evaluation.

What's Implemented (50%+ of pipeline)

- Backbone: DeiT-Tiny from scratch — Patch embedding (conv stride = patch size), positional embeddings with dynamic resize, Multi-head self-attention, GELU MLP, LayerNorm, stochastic depth
- Task head: lightweight upsampling decoder for binary segmentation
- Training: AMP-optional loop (Windows-safe), AdamW, BCE/CE losses, checkpointing
- Evaluation: single-batch sanity check + mask resizing for shape-safety
- Data: PyTorch Dataset + loaders; synthetic toy data generator committed

Key files

```
datasets/tiny_seg.py
models/vit.py
models/heads.py
utils/{metrics.py, augment.py}
train.py
eval.py
configs/imagenette_deit_tiny.yaml
scripts/make_toy_sample.py
```

Dataset

Toy Traversability Sample (committed)

200 train / 50 val images, 160×160 RGB with binary masks (traversable = white). Generated by scripts/make_toy_sample.py; stored at data/sample/. Used to verify end-to-end training & IO before switching to real data.

Planned real data

Freiburg Forest (annotated). A small slice will be converted into the same folder structure via scripts/make_sample_from_freiburg.py and placed under data/sample/ (raw data stays git-ignored).

How to Run (Windows, venv)

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

```
.\.venv\Scripts\python.exe .\train.py --config .\configs\imagenette_deit_tiny.yaml --epochs 2 --save_dir .\runs
```

```
.\.venv\Scripts\python.exe .\eval.py --checkpoint .\runs\deit_tiny.ckpt
```

```
# or
```

```
.\.venv\Scripts\python.exe .\eval.py --checkpoint .\runs\deit_tiny_last.ckpt
```

Config (summary)

img_size: 160, num_classes: 2, batch_size: 64, epochs: 10 (CLI overrides to 2), lr: 3e-4, weight_decay: 0.05, patch: 16, embed_dim: 192, depth: 12, heads: 3

Current Results (intermediate)

Environment: Python 3.12.0 (Windows), CPU; requirements locked at reports/requirements_lock.txt.

Training log (2 epochs on toy):

Epoch 1/2 - val Dice=0.0000 mIoU=0.0000

Saved best: runs\deit_tiny.ckpt

Epoch 2/2 - val Dice=0.0000 mIoU=0.0000

Saved best: runs\deit_tiny.ckpt

Saved last: runs\deit_tiny_last.ckpt

Eval sanity check:

Eval OK. Batch mask mean: ~0.239

Implementation Notes & Design Choices

- Shape safety: logits are resized to target H×W before loss/metrics to accommodate feature-map scale.
- Windows safety: num_workers=0 and if __name__ == '__main__': guards avoid multiprocessing issues.
- DeiT specifics: stochastic depth schedule across blocks; positional embeddings dynamically resampled for arbitrary input sizes.

Next Steps (toward final)

- Toy set learning curve: Train 15–20 epochs, set batch_size: 32 → expect Dice > 0 quickly.
- Switch to Freiburg slice: Use scripts/make_sample_from_freiburg.py to build data/sample/ from raw.
- Baselines & ablations: Tiny CNN vs DeiT-Tiny; patch size 8 vs 16; embed_dim 96/192.
- Quality & visuals: Save Dice/mIoU per epoch (reports/metrics.csv), plot curves; add qualitative masks.
- Paper fidelity: Add distillation head (teacher logits) to mirror DeiT's data-efficiency setting (optional).
- Deployment: Export ONNX; onnxruntime CPU inference demo.

Risks & Mitigations

- Compute limits (CPU/Colab): keep tiny configs; use mixed precision on GPU; early-stop via val mIoU.
- Dataset size: keep committed sample small; raw Freiburg remains ignored to keep repo lightweight.
- Reproducibility: lockfile at reports/requirements_lock.txt; config-driven runs; checkpoints in runs/.

Artefacts

- Code (train/eval/models/datasets/scripts)
- Configs
- Committed sample data (data/sample/)
- Checkpoints (runs/deit_tiny.ckpt, runs/deit_tiny_last.ckpt) — generated locally
- reports/requirements_lock.txt
- This report